



PRESIDENCY COLLEGE

(AUTONOMOUS)

AFFILIATED TO BENGALURU CITY UNIVERSITY, APPROVED BY AICTE, DELHI & RECOGNISED BY THE GOVT. OF KARNATAKA

RE-ACCREDITED BY NAAC WITH 'A++' GRADE

Unit 1: Stacks and Queues

Notes Prepared by Prof. Veena S Badiger

Introduction to stacks and queues Implementation of stacks and queues using arrays and linked lists Applications of stacks and queues Evaluating postfix expressions Implementing recursive algorithms using stacks Implementing job scheduling using queues Time and space complexity analysis of stack and queue operations Implement a stack using an array Implement a queue using a linked list Evaluate a postfix expression using a stack

Introduction and Overview:

What is Data Structure?

The data structure name indicates itself that organizing the data in memory. There are many ways of organizing the data in the memory as we have already seen one of the data structures, i.e., array in C language. Array is a collection of memory elements in which data is stored sequentially, i.e., one after another. In other words, we can say that array stores the elements in a continuous manner. This organization of data is done with the help of an array of data structures. There are also other ways to organize the data in memory.

The organized collection of data is known as data structure. The data structure deals with the study of how data is organized in the memory.

Data Structure = Organized Data + Operations

Applications of data structures:

- It provides a function that can be used to retrieve the individual data elements.
- Data structures helps to describe the operation that must be performed on the logically related data elements, the operations such as creating, displaying, inserting, deleting, retrieving etc.
- Data structure enables to devise the methods of representing the data elements in the memory of the computers, that will reduce the loss of fragmentation and also allows to select the memory configuration or storage structures.
- Data structures gives the freedom to the programmer to decide any type of language that best suits for a particular problem.
- Data structures are used to solve real time problems.
- Data structures are used in operating system memory management algorithms.

Elementary data organization:

Data represents a single value or a set of values assigned to entities. Data item refers a single or group of values with in the data .An entity is a thing that has some properties which can take values. Processed or meaning full data is called information. This is used for taking some action.

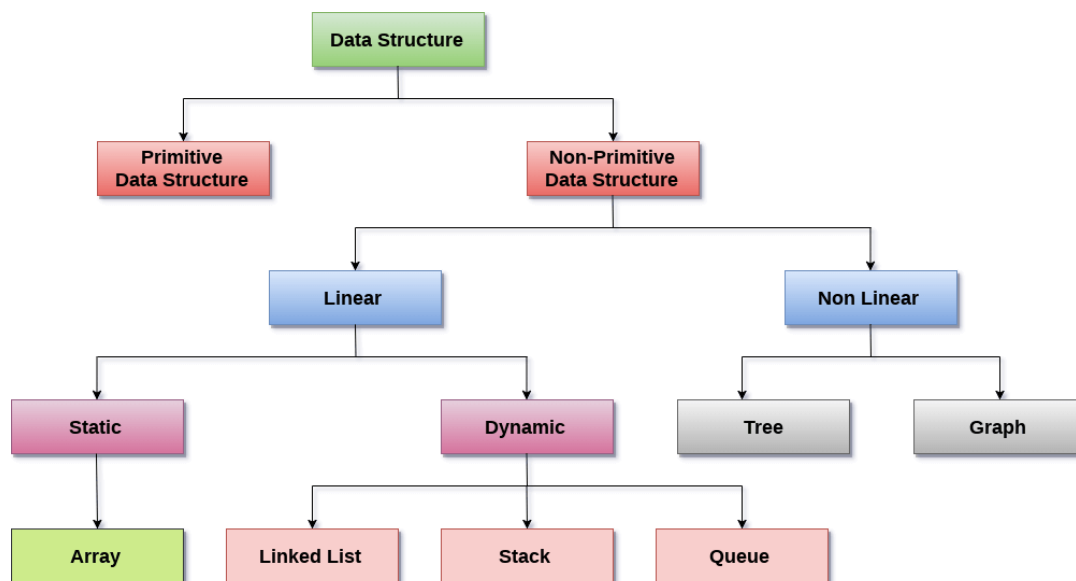
Information = Instruction + Data

Data is a part of information and a value or set of values represented by data is stored temporarily in the programs data area or stored permanently on a file in storage devices

Classification of data structures:

There are two types of data structures:

- Primitive data structure
- Non-primitive data structure



Primitive Data structure

The primitive data structures are primitive data types. The int, char, float, double, and pointer are the primitive data structures that can hold a single value.

Non-Primitive Data structure

The non-primitive data structure is divided into two types:

- Linear data structure
- Non-linear data structure

Linear data structure:

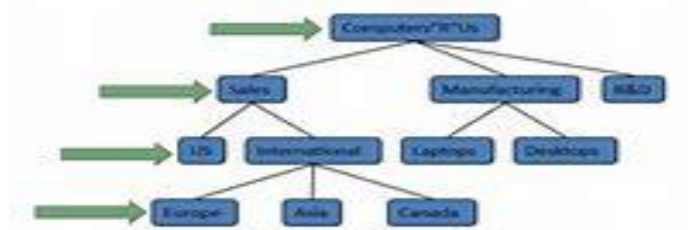
- Linear data structure: in this the data is stored in linear manner i.e it establishes the adjacency relationship between elements.
- A Linear data structure have data elements arranged in sequential manner and each member element is connected to its previous and next element. This connection helps to traverse a linear data structure in a single level and in single run.



Non Linear data structure:

- Establishes the relationship other than the adjacency.
- Non-linear data structure, data elements are hierarchically connected and are present at various levels.

Parent-child relationship



1. STACK(LIFO- Last in First Out):

Stores a set of elements in a particular order. Stack is a linear data structure in which data is inserted and deleted at one end called as top of stack. Stack principle: LAST IN FIRST OUT(LIFO)

	Stack	LIFO				top
address	100	101	102	103	104	105
	11	12	13	14	15	16
	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]

2. QUEUE(FIFO- First in First out)

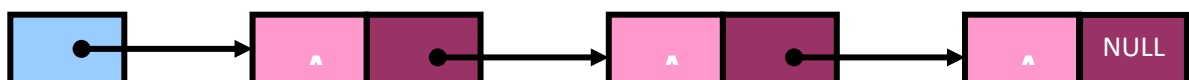
A queue is like a line of people waiting for a bank teller. The queue has a front and a rear. Insertion of element takes place at rear end and deletion takes place at front end. Types-Linear queue, Circular queue, Priority queue, Dqueue.

	Queue	FIFO				rear
	Front					
address	100	101	102	103	104	
	11	12	13	14	15	
	a[0]	a[1]	a[2]	a[3]	a[4]	

3. LINKED LIST

They grow and shrink one element at a time, normally without some of the inefficiencies of arrays.

A linked list is a series of connected nodes. Each node contains- data and address of next node

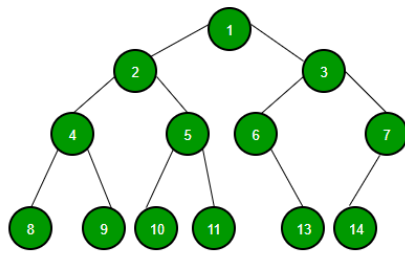


4. TREES

Trees are multilevel data structures with a hierarchical relationship among its elements known as nodes. The bottommost nodes in the hierarchy are called **leaf node** while the topmost node is called **root node**. Each node contains pointers to point adjacent nodes.

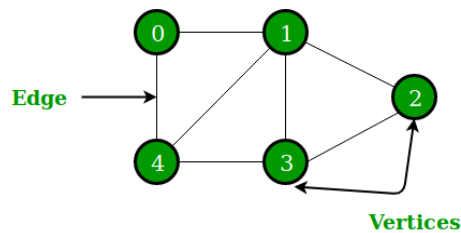
Tree data structure is based on the parent-child relationship among the nodes. Each node in the tree can have more than one children except the leaf nodes whereas each node can have at most one parent except the root node. Trees can be classified into many categories as Binary tree, Binary search tree, Heap tree, Strictly binary tree, Complete binary tree, Almost complete binary tree.

Operations on trees are -Insertion, Deletion, Searching, Traversals(preorder, inorder, post order)



5. Graphs

Graphs can be defined as the pictorial representation of the set of elements (represented by vertices) connected by the links known as edges. A graph is different from tree in the sense that a graph can have cycle while the tree can not have the one.



Operations on data structures

1) Traversing: Every data structure contains the set of data elements. Traversing the data structure means visiting each element of the data structure in order to perform some specific operation like searching or sorting.

Example: If we need to calculate the average of the marks obtained by a student in 6 different subject, we need to traverse the complete array of marks and calculate the total sum, then we will divide that sum by the number of subjects i.e. 6, in order to find the average.

2) Insertion: Insertion can be defined as the process of adding the elements to the data structure at any location.

If the size of data structure is **n** then we can only insert **n-1** data elements into it.

3) Deletion: The process of removing an element from the data structure is called Deletion. We can delete an element from the data structure at any random location.

If we try to delete an element from an empty data structure then **underflow** occurs.

4) Searching: The process of finding the location of an element within the data structure is called Searching. There are two algorithms to perform searching, Linear Search and Binary Search. We will discuss each one of them later in this tutorial.

5) Sorting: The process of arranging the data structure in a specific order is known as Sorting. There are many algorithms that can be used to perform sorting, for example, insertion sort, selection sort, bubble sort, etc.

6) Merging: When two lists List A and List B of size M and N respectively, of similar type of elements, clubbed or joined to produce the third list, List C of size (M+N), then this process is called merging

Stack

Stack is **non-primitive linear data structure** into which, new items may be inserted and from which items may be deleted. In this data structure, the insertion and deletion of the elements are carried out at one end and is called as „**top**“ of the stack. In stack, the element last inserted will be the first to be deleted. Hence, stack is known as **Last In**

First Out (LIFO) structure.

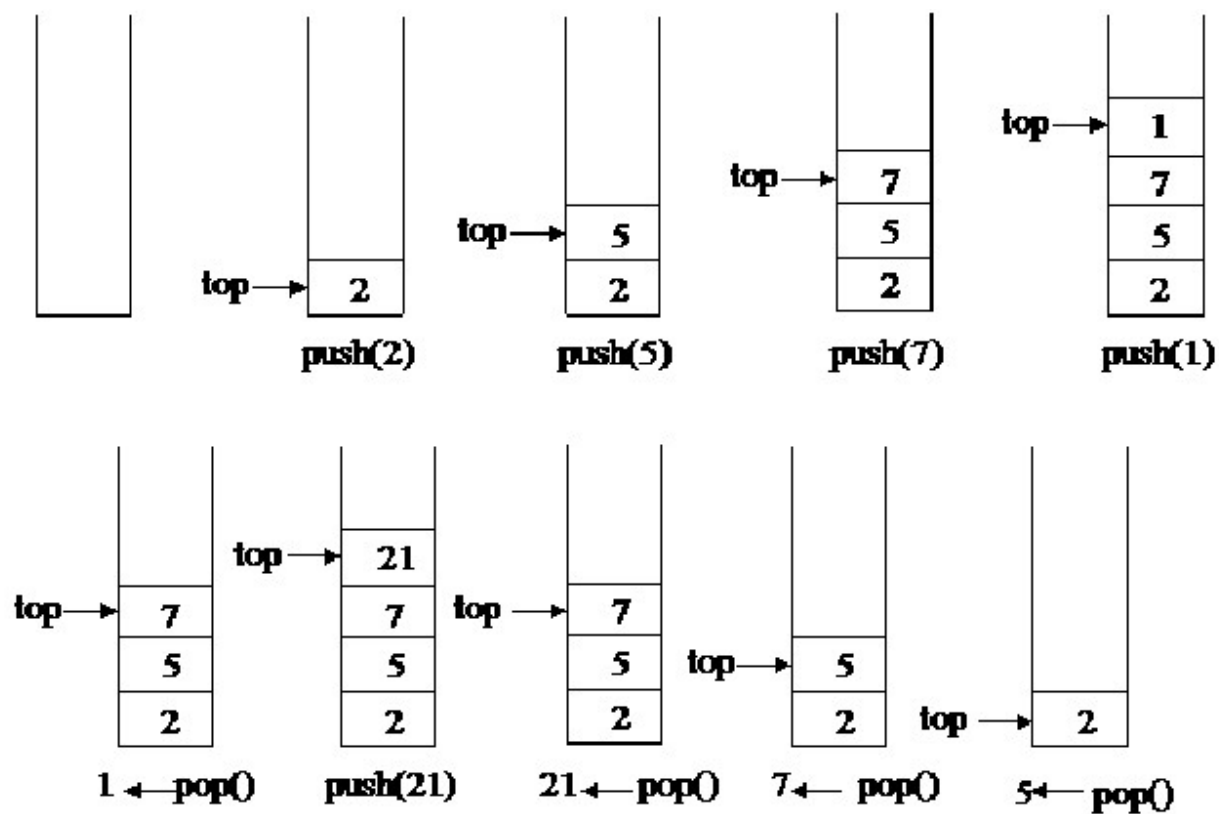
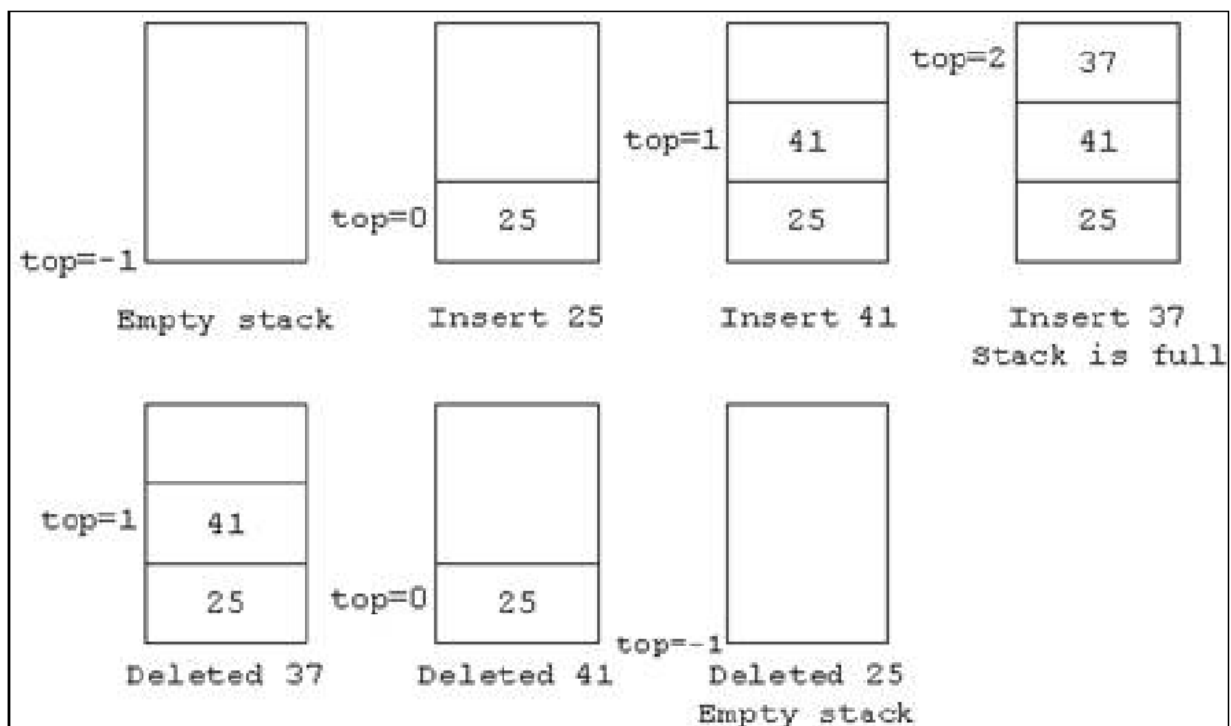
For example, consider the task of keeping books on a table one above the other. When a person wants to take the book, he has to take the book which was kept last. Thus, the first book kept on the table will be the last book to be taken out.

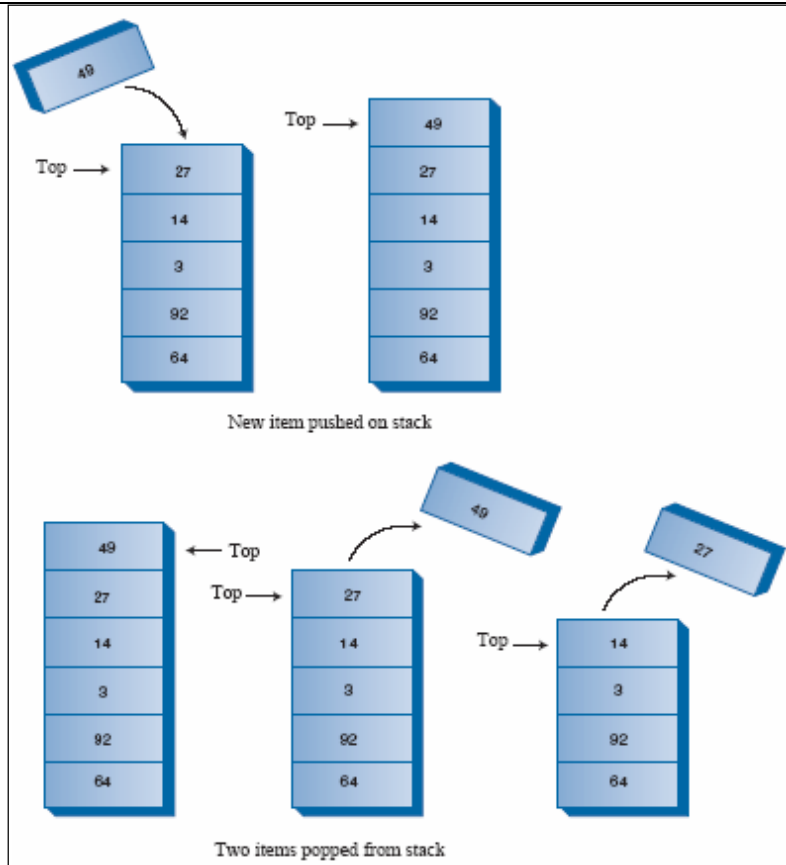
In the study of data structures, **insertion** of new item into the stack is called as **push operation**; whereas, **deletion** of an item at the top of the stack is **pop operation**. When the stack is full, we can't insert any more elements. This situation is called as **stack overflow**. Similarly, when stack is empty, we can't delete element from it. This condition is known as **stack underflow**.

An integer variable top is normally used for **denoting current status of the stack** – that is, the value of top gives the number of items of the stack. For programming purpose, an empty stack is denoted by setting the value of **top as -1**. Each time the push operation is encountered, the top will be incremented. When the pop operation is done, top will be decremented. Usually, we define the size of the stack at the beginning.

For example, stack of 5 elements, stack of 10 elements etc. When the value of top becomes -1 during deletion, it is stack underflow. When the top reaches the predefined size, it is stack overflow.

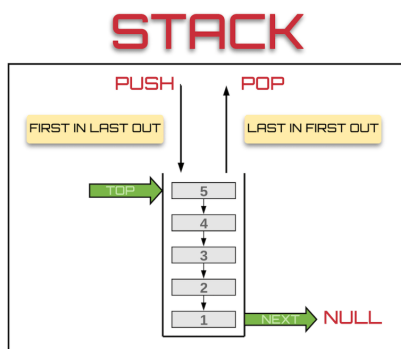
The figures show Demo of stack operations.





Definition of stack:

“Stack is a linear data structure in which data is inserted and deleted at one end called as top of stack”. A stack is another version of an array. Stores a set of elements in a particular order. It works on the principal of LIFO i.e last in and first out. In stack last item inserted in the stack will be first item to be removed.



There are many real-life examples of a stack. Consider an example of plates stacked over one another in the canteen. The plate which is at the top is the first one to be removed, i.e. the plate which has been placed at the bottommost position remains in the stack for the longest period of time. So, it can be simply seen to follow LIFO (Last In First Out)/FILO (First In Last Out) order.

Array implementation of stack:

Stack can be represented using arrays or linked list. Like array stack is also collection of same type of items. We can take array for implementing a stack.

An array is used to store an ordered list of elements. Using an array for representation of stack is one of the easy techniques to manage the data. But there is a major difference between an array and a stack.

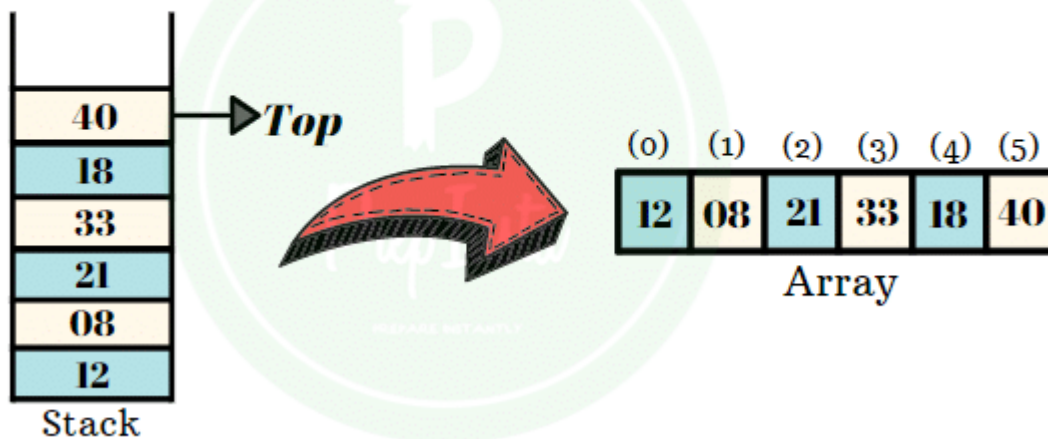
- Size of an array is fixed.
- While, in a stack, there is no fixed size since the size of stack changed with the number of elements inserted or deleted to and from it.
- Allocate the memory block of sufficient size prior to the operation. It is usually suggested to declare the array to be large enough without wasting too much of memory space.

Despite the difference, an array can be used to represent a stack by taking an array of maximum size; big enough to manage a stack.

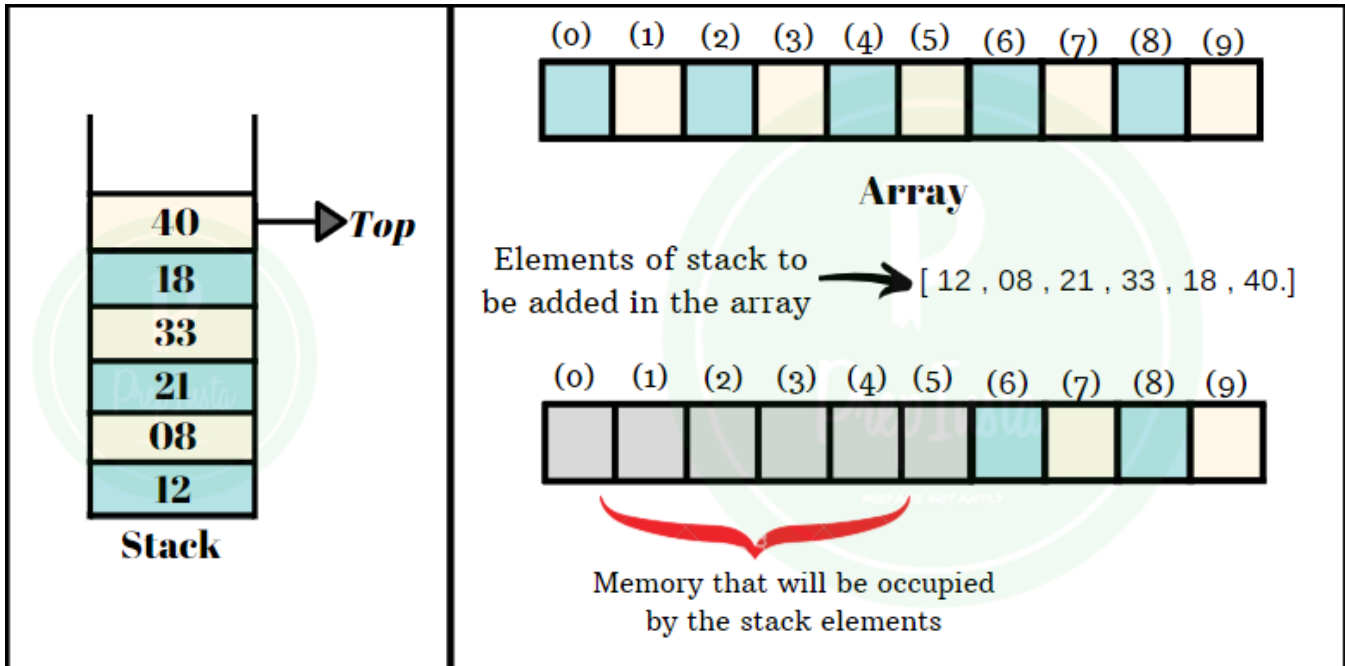
Example1 to show array representation of stack:



Stack as an Array



We are given a stack of elements: 12 , 08 , 21 , 33 , 18 , 40.



Example2:

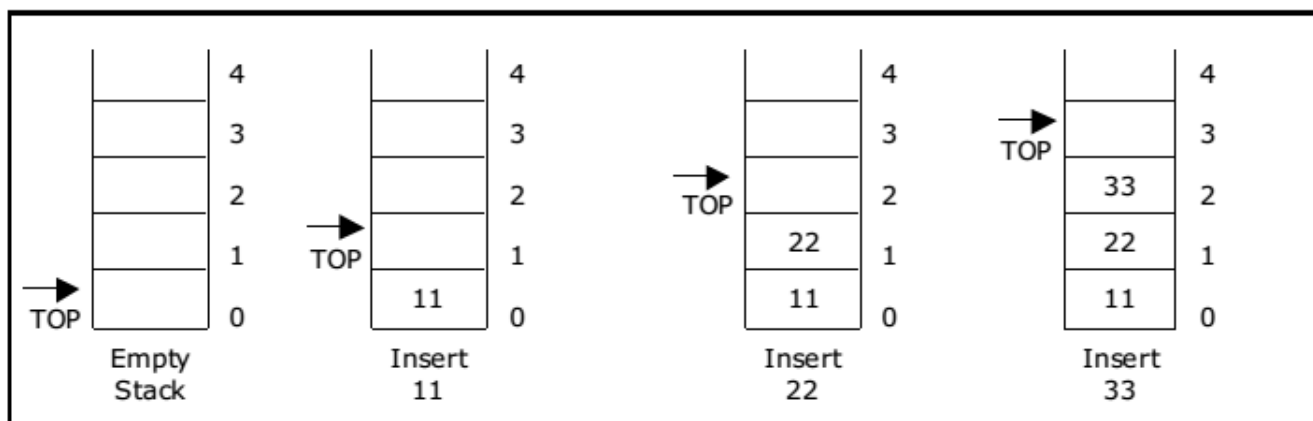


Figure 4.1. Push operations on stack

Stack as ADT(2 marks)

- **push()** – Insert an element at one end of the stack called top.
- **pop()** – Remove and return the element at the top of the stack, if it is not empty.
- **isEmpty()** – Return true if the stack is empty, otherwise return false.
- **isFull()** – Return true if the stack is full, otherwise return false.
- **Count()**– count the number of elements in stack.
- **Traverse()**– accessing all the elements in the stack.

Operations on stack: (5 marks)

- Push operation

Push an element on top of the stack. If the stack is full, then PUSH operation is not possible. Initially top = -1

Step 1 – Checks if the stack is full. If the stack is full, produces an error and exit.

Step 2 – If the stack is not full, increments top to point next empty space.

Step 3 – Adds data element to the stack location, where top is pointing.

Step 4 – Returns success

Algorithm: push(stack, top, maxstk, item)

Step1: [Overflow check]

```
    if (top == maxstk - 1) then
        write ("stack overflow ")
    return;
```

Step2: [increment top value]

```
    top = top + 1
```

Step 3: [insert the item at the top]

```
    stack[top] = item
```

Step 4: return;

- **Pop operation**

Pop an element from the stack (Top element). If the stack is empty, then POP operation is not possible

Step 1 – Checks if the stack is empty. If the stack is empty, produces an error and exit.

Step 2 – If the stack is not empty, accesses the data element at which top is pointing. □

Step 3 – Decreases the value of top by 1.

Step 4 – Returns success.

Algorithm: pop(stack, top, item)

Step1: [Underflow check]

```
    if (top == -1) then
        write ("stack underflow ")
    return;
```

Step2: [delete the item]

```
    item = stack[top]
```

Step 3: [decrement top value]

```
    top = top - 1
```

Step 4: return;

- **DISPLAY operation**

Displaying all elements in the stack

Algorithm: Display()

```
{
    if(top == -1)
    {
        Write " Stack is empty
    return
```

```

    }
    else
    {
        for(i=top;i>=0;i--)
            Print s[i]
    }
}

```

- **Isempy operation:**

Checks whether stack is empty. Returns true if the stack is empty

Algorithm: isempty()

```

{
    if(top== -1)
        return 1
    else
        return 0
}

```

- **Isfull operation**

Checks whether stack is full. Returns true if the stack is full.

Algorithm: isfull()

```

{
    if(top==maxsize-1)
        return 1
    else
        return 0
}

```

C program to implement stacks using array: (lab program1)

```

#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#define max 3
int top = -1;
int stack[max];
int isfull();
int isempty();

void push( )
{
    int item;
    if(isfull())
    {
        printf("\n stack overflow");
        getch();
    }
    else
    {
        printf("Enter an item to be pushed");
        scanf("%d", &item);
        top++;
        stack[top]=item;
    }
}

```

```

        if(top==max-1)
            printf("stack is full\n");
    }
}
void pop( )
{
    int item;
    if(isempty())
    {
        printf("\n stack underflow");
        getch();
    }
    else
    {
        item= stack[top];
        top--;
        printf("Popped element = %d", item);
        if (top==-1)
            printf("\nStack is empty");
    }
    getch();
}

int isempty( )
{
    if ( top== -1)
        return 1;
    else
        return 0;
}

int isfull( )
{
    if(top==max-1)
        return 1;
    else
        return 0;
}

void display( )
{
    int i;
    if(isempty())
    {
        printf("stack is empty\n");
    }
    else
    {
        printf("stack elements are:\n");
        for(i=top;i>=0;i--)

```

```

        printf("%d\t", stack[i]);
    }
    getch();
}

void main()
{
    int ch;
    //clrscr();
    while (1)
    {
        printf("\n Stack menu\n");
        printf("1. Push\n");
        printf("2. Pop \n");
        printf("3. Display \n");
        printf("4. Exit \n");
        printf("Enter you choice\n");
        scanf("%d", &ch);

        switch(ch)
        {
        case 1: push();
            break;
        case 2: pop();
            break;
        case 3: display();
            break;
        case 4: exit(0);
            break;
        default :printf("Invalid choice");
            break;
        }
        //clrscr();
    } //end of while loop
}

```

What are the various applications of Stack?

- Recursion
- Reversal of a string
- Checking the parenthesis matching
- Postfix expression evaluation
- Infix to postfix conversion
- Infix to prefix conversion

Application- Reversal of string using stack

- Push each character of the string on the stack, when the whole string is pushed on the stack then popping of the characters from the stack is done till stack is empty.

Algorithm: reverse_string_stack(str[],s[])

//input: str[] is string and s[] is stack of characters

//output: reverse of the string str[]

```

for(i=0 to length(str))
    push(str[i])
for(i=0 to length(str))
    str[i]=pop()
print str

```

example: str[] "data"

character read	stack content	operation
Initially	empty	
D	d	push(d)
A	a	push(a)
T	t	push(t)
A	a	push(a)
\0	t	pop()
	a	pop()
	d	pop()
	empty	pop()

Polish Notations

Polish mathematician Jan Lukasiewicz introduced a new technique to represent arithmetic expressions. Arithmetic Expressions involving operators and operands can be classified three types depending on how the operator is placed between operands.

1. Infix notation
2. Postfix notation
3. Prefix notation

INFIX

To add A, B, we write A+B

To multiply A, B, we write A*B

The operators ('+' and '*') go **in between the operands** ('A' and 'B'). This is "*Infix*" notation.

PREFIX

Instead of saying "A plus B", we could say "add A,B " and write + A B

"Multiply A,B" would be written *A B

This is *Prefix* notation.

POSTFIX

Another alternative is to put the operators after the operands as in A B +

And A B *

This is *Postfix* notation.

Examples:



Example

infix

$(a + b) * c$
 $a + (b * c)$

postfix

$a b + c *$
 $a b c * +$

prefix

$* + a b c$
 $+ a * b c$

Infix form : $\langle identifier \rangle \langle operator \rangle \langle identifier \rangle$

Postfix form : $\langle identifier \rangle \langle identifier \rangle \langle operator \rangle$

Prefix form : $\langle operator \rangle \langle identifier \rangle \langle identifier \rangle$

Operator Priorities/Precedence rule:

Postfix conversion operation depends on operator priority are given below
 Operators are taken out of the stack as long as the in-stack priority is greater than or equal to the in-coming priority of the new operator

Highest level	$\wedge$ (exponent)
Next higher level	$*$ / (multiplication & division)
Lowest level	$+$ - (addition & subtraction)

Application- Convert infix expression to postfix (2marks)

Example 1:

$((a+b)*c)$
 $(ab+ * c)$
 $ab+c*$

Example 2:

$(a+b*c)$
 $a + bc*$
 $abc*+$

Example3:

$((A+(B^{\wedge}C)-D)) * (A/C)$
 $((A + BC^{\wedge} - D)) * AC/$
 $((ABC^{\wedge}+ - D) * AC/$
 $(ABC^{\wedge}+D-) * AC/$
 $ABC^{\wedge}+D-AC/*$

Example4:

$A + B * C + (D * E + F) * G$
 $A + B * C + (DE* + F) * G$
 $A + B * C + DE*F+ * G$
 $A + B * C + DE*F+G*$

$A + BC^* + DE^*F + G^*$

$ABC^{*+} + DE^*F + G^*$

$ABC^{*+} DE^*F + G^{*+}$

Convert infix expression to postfix (5marks)

Important rules to be followed for conversion of infix to postfix

Symbol scanned	Stack and operation performed
(	Push (in stack
)	Pop until top of stack is (
Operator is equal precedence with top of stack (+,-)	Operator is equal precedence with symbol scanned (+ , -) pop from stack and push the operator scanned
Operator (low precedence +,-) with top of stack	Operator (high precedence *,/) with symbol scanned, then perform pop and push the operator scanned
Operator (high precedence *, / , ^) with top of stack	Operator (low precedence +, -) with symbol scanned, then push the operator scanned

Example 1:

$(A+B^*C)$

Symbol scanned	Stack	P(postfix expression)
	#	
(	#(	
A	#(	A
+	#(+	a
B	#(+	Ab
*	#(+*	Ab
C	#(+*	Abc
)	#	Abc*+

Example 2:

$A + (B + C)$

Symbol scanned	Stack	Postfix
A		A
+	+	A
(	+(	A
B	+(	AB
+	+(+	AB
C	+(+	ABC
)	+	ABC+
No symbols	EMPTY	ABC++

Example3:

$A+B^*C$

Symbol scanned	Stack	postfix
----------------	-------	---------

A	#	A
+	#+	A
B	#+(small)	AB
(big)	#+	AB
C	#+*	ABC
No symbol	EMPTY	ABC*+

Example 4:

A*B/C

SYMBOL SCANNED	STACK	POSTFIX
A	#	A
*	#*	A
B	#*	AB
/	#/	AB*
C	#/	AB*C
NO SYMBOL	#	AB*C/

Example 5:

A+ (B*C-(D/E^F)*G)*H

Symbol scanned	Stack	Postfix expression
	#	
A	#	A
+	#+	A
(	#+(	A
B	#+(	AB
*	#+(*	AB
C	#+(*	ABC
-	#+(-	ABC*
(	#+(-(	ABC*
D	#+(-(	ABC*D
/	#+(-(/	ABC*D
E	#+(-(/	ABC*DE
^	#+(-(/^	ABC*DE
F	#+(-(/^	ABC*DEF
)	#+(-	ABC*DEF^/
*	#+(-*	ABC*DEF^/
G	#+(-*	ABC*DEF^/G
)	#+	ABC*DEF^/G*-
*	#+*	ABC*DEF^/G*-
H	#+*	ABC*DEF^/G*-H
No symbol to scan	#	ABC*DEF^/G*-H*+

Algorithm to convert infix expression to postfix:

Algorithm: convert infix_postfix()

{

- 1) Push "(" onto STACK, and add ")" to the end of Q
- 2) scan Q from left to right and repeat steps 3 to 6 for each element of Q until the stack is empty.
- 3) if an operand is encountered add it to P.

4) if a left parenthesis is encountered push it onto the stack

5) **if** an operator is encountered , then

(a) Repeatedly pop from stack and add to P each operator (on top of the STACK)

which has same precedence as or higher precedence than the operator encountered.

(b) Push the encountered operator to the stack

[end of IF structure]

if a right parenthesis is encountered, then:

(a) repeatedly pop from the stack and add to P each operator (on top of the STACK) until a left parenthesis is encountered .

(b) remove the left parenthesis (do not add left parenthesis to P).

[End of IF structure]

[End of step 2 loop]

7) Exit.

}

C Program to convert infix expression to postfix:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<math.h>
#include<stdlib.h>
#include<ctype.h>
#define max 50
```

```
char s[max];
```

```
char pop();
```

```
int top=0;
```

```
int precedence (char elem);
```

```
void push(char elem);
```

```
void main( )
```

```
{
```

```
    char infix[max], suffix[max], ch, elem;
```

```
    int i=0, j=0;
```

```
    //clrscr();
```

```
    printf("Enter infix expression \n");
```

```
    gets(infix);
```

```
    push('#');
```

```
    for(i=0;i<strlen(infix);i++)
```

```
    {
```

```
        ch=infix[i]; //scan the infix expr until string length reached
```

```
        if(isalnum(ch)) //check for alphanumeric
```

```
            suffix[j++]=ch;
```

```
        else if(ch=='(')
```

```
            push(ch);
```

```
        else if(ch==')')
```

```
        {
```

```
            while(s[top]!='(')
```

```

        suffix[j++]=pop();
        elem=pop();
    }
    else
    {
        while(precedence(s[top])>=precedence(ch))
            suffix[j++]=pop();
        push(ch);
    }
}
while(s[top]!='#')
    suffix[j++]=pop();
suffix[j]='\0';
printf("\n the suffix expression is :%s",suffix);
getch();
}
int precedence (char elem)
{
    switch(elem)
    {
        case '+':
        case '-': return (1);
        case '*':
        case '/': return (2);
        case '^': return (3);
        case '(':
        case '#': return(0);
    }
}

void push(char elem)
{
    ++ top;
    s[top]=elem;
}

char pop()
{
    char elem;
    elem =s[top];
    --top;
    return (elem);
}

```

Application- Evaluating POSTFIX expression Algorithm (5marks)

Step 1: Add ‘#’ at the end of p.

step 2: Repeat scanning P from left to right and repeat step 3 and 4 for each element of p until "#" is encountered.

Step 3: If an operand is encountered, push it onto STACK.

Step 4: If an operator @ is encountered:

- 1) Remove the two top elements of STACK.
- 2) If A is the top element, B is the next top element.
 - a) Evaluate $B @ A$. [NOT $A @ B$]
 - b) Push the result of (2) of step 4 back onto STACK.

Step 5: Evaluated value is equal to the top of the STACK.

Step 6: Exit

Examples:

Symbol	Stack S	Action
3	3	Push 3 into S
5	3, 5	push 5 into S
7	3, 5, 7	push 7 "
4	3, 5, 7, 4	push 4 "
-	3, 5, 3 →	Pop 4, pop 7 then pop 7-4 push
2	3, 5, 3, 2	Push 2
^	3, 5, 9	pop 2, pop 3, then pop 3^2 push
*	3, 45	Push $5 * 9$
+	48	Push $45 + 3$

2. 2 3 4 + * 5 *		EX i) 12 + 3 - 21 + 3 = ii) 6 3 2 - 5 2 1 + 3 *
2	2	
3	2, 3	
4	2, 3, 4	
+	2, 7	pop 4 & pop 3. push 3+4
*	14	pop 7, pop 2. push 2*7
5	14, 5	pop 5 & pop 14. push 70

Application- Conversion of infix expression to prefix Algorithm (5marks)

To convert an infix to postfix expression following steps are applied:

- Step 1: Reverse the infix expression i.e $A+B*C$ will become $C*B+A$. Note while reversing each '(' will become ')' and each ')' becomes '('.
- Step 2: Obtain the “nearly” postfix expression of the modified expression i.e $CB*A+$.
- Step 3: Reverse the postfix expression. Hence in our example prefix is $+A*BC$.

Example to convert infix to prefix without stack 2 marks:

$(a+b*c)$

$a + *bc$

$+a*bc$

Example to convert infix to prefix using stack 5 marks:

$(a+b*c)$

Reverse the given expression

$)c*b+a($

Symbol scanned	stack	Prefix
	#	
)	#)	
C	#)	c
*	#)*	c
B	#)* (high)	cb
+ (low)	#)+	Cb*
A	#)+	Cb*a
(	#	Cb*a+
No symbol	#	Cb*a+

The prefix expression is reverse($cb*a+$)

$+a*bc$

Algorithm to convert infix expression to prefix:(5marks)

Step 1: Start

Step 2: Reverse the infix expression

Step 3: Obtain the postfix form

3.1:Start reading the infix expression from left to right.

3.2: Repeat Step 3.3 to 3.6 for each element until the Stack is empty.

3.3: If we scan a operand we output it, print it.

3.4: Else,

3.4.1: If the scanned operator is greater in precedence than the operator in the stack or if the stack is empty or the stack contains a "(", push it.

3.4.2: Else,

Pop all the operators having greater or equal precedence that of the scanned operator. After doing that Push the scanned operator to the stack. In case there is a parenthesis while popping then stop and push the scanned operator in the stack.

3.5: If a '(' is encountered, push it onto Stack.

3.6: If a ')' is encountered, repeatedly pop from Stack and output it until a '(' is encountered.3.7:
The output is printed in postfix notation.

Step 4: The expression formed is in postfix form, reverse it and print it,
this is the prefix form

Step 5: Stop

C program to convert infix expression to prefix:(5marks)

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<stdlib.h>
#include<ctype.h>
#define max 50

char s[max];
char pop();
int top=0;
int precedence (char elem);
void push(char elem);
char pop();
void main( )
{
    char infix[max], prefix[max], ch, elem;
    int i=0, j=0;
    clrscr();
    printf("Enter infix expression \n");
    gets(infix);
    strrev(infix);
    push('#');
    for(i=0;i<strlen(infix);i++)
    {
        ch=infix[i];           //scan the infix expr until string length reached
        if(isalnum(ch))        //check for alphanumeric
            prefix[j++]=ch;
        else if(ch==' ')
            push(ch);
        else if(ch=='(')
        {
            while(s[top]!='')
                prefix[j++]=pop();
            elem=pop();
        }
        else
        {
            while(precedence(s[top])>=precedence(ch))
                prefix[j++]=pop();
            push(ch);
        }
    }
    while(s[top]!='#')
        prefix[j++]=pop();
}
```

```

        prefix[j++]=pop();
    prefix[j]='\0';
    printf("\n the prefix expression is :%s",strrev(prefix));
    getch();
}
int precedence (char elem)
{
    switch(elem)
    {
        case '+':
        case '-': return (1);
        case '*':
        case '/': return (2);
        case '^': return (3);
        case ')':
        case '#': return(0);
    }
}

void push(char elem)
{
    ++ top;
    s[top]=elem;
}

char pop()
{
    char elem;
    elem =s[top];
    --top;
    return (elem);
}

```

Application – Recursion using Stack

Meaning

Recursion is a technique of defining something in terms of itself. If a function calls itself then it is known as recursive function. If the function calls itself directly, then it is known as direct recursion.

Definition: (1mark)

Recursion is a process of calling the function repeatedly in terms of itself. The repeated calling is terminated by the specific condition.

Example:

```

void myfun()
{ ----
----
myfun();
----
----
}

```

Properties of recursive functions

- A recursive solution must contain a **base case**.
- A recursive solution must contain a **recursive case**.
- A recursive solution must make progress toward the base case.

Types of Recursion:

1. **Direct recursion:** in the direct recursion the function calls itself repeatedly until certain condition is satisfied.

```
fact(int n )
{
    if(n==1)----- base case or termination case
        return(1);
    else
        return(n*fact(n-1));----- recursive call
}
```

2. **Indirect recursion:** in indirect recursion, the function calls another function which eventually causes the same function to be called.

```
Func(int a)
{
    func1(b);
}
Func1(int b)
{
    func(b-1);
}
```

Advantages of Recursion: (2marks)

1. Recursion reduces the complexity of the problem
2. Recursion allows the user to write much simpler and more elegant programs.
3. Program implemented by recursion will be smaller in length.
4. The recursion programs can have any number of nesting levels
5. The recursion technique is more natural and compact

Example1: To find sum of natural numbers using recursion (1+2+3-----)

```
#include<stdio.h>
int add(int n);
int main()
{
    int n;
    printf("Enter an positive integer: ");
    scanf("%d",&n);
    printf("Sum = %d",add(n));
    return 0;
}
int add(int n)
{
    if(n==0)----base case
        return (0);
    else
```



```

return (n+add(n-1)); /* recursive call */
}

```

Tracing:

- return n+add(n-1);
- add(4)
- Return 4+add(3)
- return 3+add(2)
- return 2+add(1)
- return 1+add(0)
- 1
- 2 + 1
- 3 + 3
-
- 4 + 6

Example2: Factorial of a given number $n! = n*(n-1)*(n-2) \dots 1$

Factorial of a non-negative integer n is defined as the product of all integers from 1 to n.

That is: $n! = n(n-1)(n-2) \dots 3.2.1$, for all $n \geq 1$ and $0! = 1$.

By definition we have, $n! = n(n-1)!$

Thus, factorial is a recursive function.

Consider one example of finding

$$5! - 5! = 5*(5-1)!$$

$$= 5*4!$$

$$= 5*4*3!$$

$$= 5*4*3*2!$$

$$= 5*4*3*2*1!$$

$$= 5*4*3*2*1*0!$$

$$= 5*4*3*2*1*1$$

$$= 120$$

This procedure can be implemented through program given below:

//FACTORIAL OF A NUMBER

```

#include<stdio.h>
int fact(int n)
{
    if (n==0)
        return 1;
    else
        return n* fact(n-1);
}
void main()
{
    int n;
    printf("Enter the value of n");
    scanf("%d", &n);
    if(n<0)
        printf("Invalid input");
    else
        printf("\nFactorial of %d is %d ", n, fact(n));
}

```

Tracing for 3!

```
n=3      fact(3)
else      return(3 *fact(2));      return(3* 2)  return(6)  6

n=2      fact(2)
else      return(2*fact(1));      return(2*1);  return(2);  2

n=1      fact(1)
else      return(1*fact(0));      return(1*1);  return(1)  1

n=0      fact(0)
if(n==0)  return(1);              return(1);
```

Example3: Fibonacci series (0 1 1 2 3 5 8-----)

A sequence of integers is called Fibonacci sequence if an sum of its immediate two predecessors.

That is:

```
f(x)=0    for x=1
f(x)=1    for x=2
f(x)=f(x-2)+f(x-1) for x>2
```

Thus the Fibonacci sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, Consider an example of finding 6th Fibonacci number.

```
f(6) = f(4) + f(5)
      = f(2) + f(3) + f(3) + f(4)
      = 1 + f(1) + f(2) + f(1) + f(2) + f(2) + f(3)
      = 1 + 0 + 1 + 0 + 1 + 1 + f(1) + (2)
      = 4 + 0 + 1
      = 5
```

The above procedure can be illustrated through the program given below:

//Finding nth Fibonacci number

```
#include<stdio.h>
int fibo(int n)
{
    if ((n==0)||(n==1))
        return 0;
    else if (n==2)
        return 1;
    return (fibo(n-1) + fibo (n-2));
}
void main()
{
    int n;
    printf("Enter value of n");
    scanf("%d", &n);
    if(n<=0)
        printf("invalid input");
    else
        printf("%d th fibonacci number is %d ", n, fibo(n));
}
```

Tracing to find 4th Fibonacci value:

```
        fib(4)
      return(1+1)
    return(  fib(3)+fib(2))

    return(1+0)
  return(  fib(2)+  fib(1))
```

Recursive tree to find 4th Fibonacci value

Example4: Towers of Hanoi

In the problem of Tower of Hanoi, there will be three poles, viz. A, B and C. Pole A (source pole) contains „n" discs of different diameters and are placed one above the other such that larger disc is placed below the smaller disc. Now, all the discs from source (A) must be transferred to destination (C) using the pole B as temporary storage.

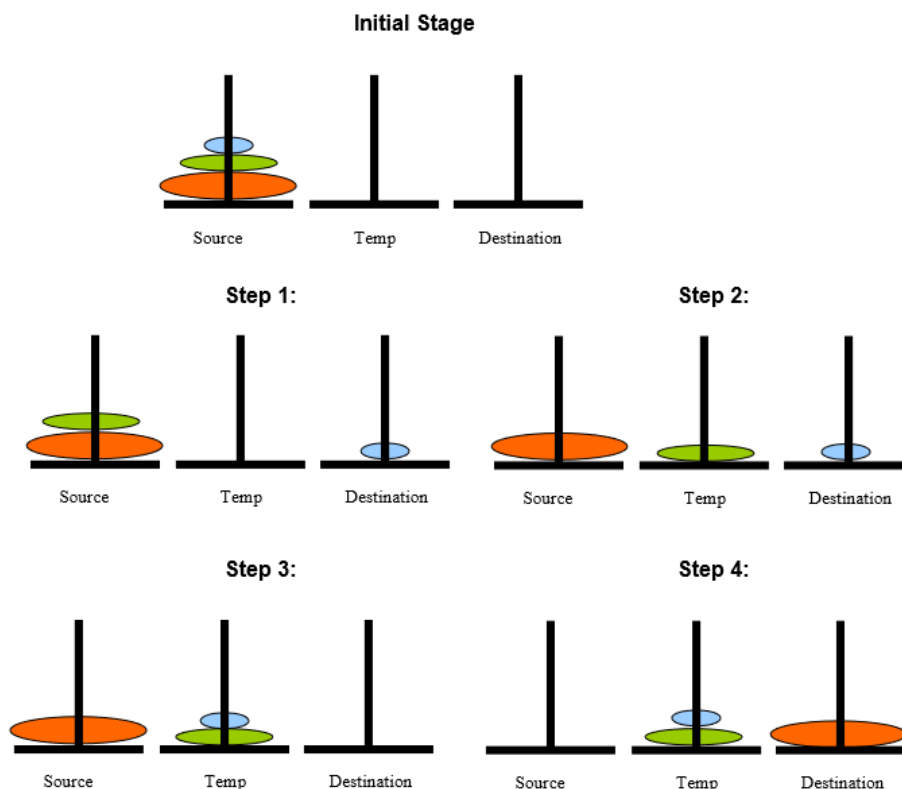
Conditions are:

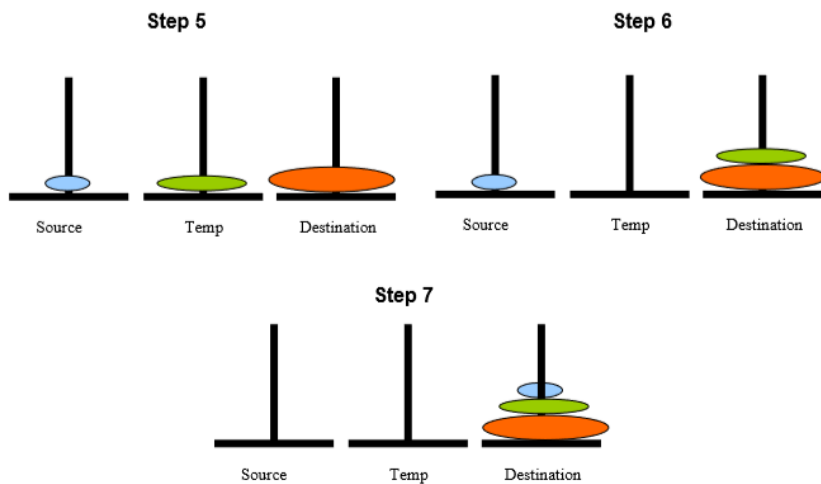
- Only one disc must be moved at a time
- smaller disc is on the top of larger disc at every step

Algorithm to move n discs from source to destination is as follows –

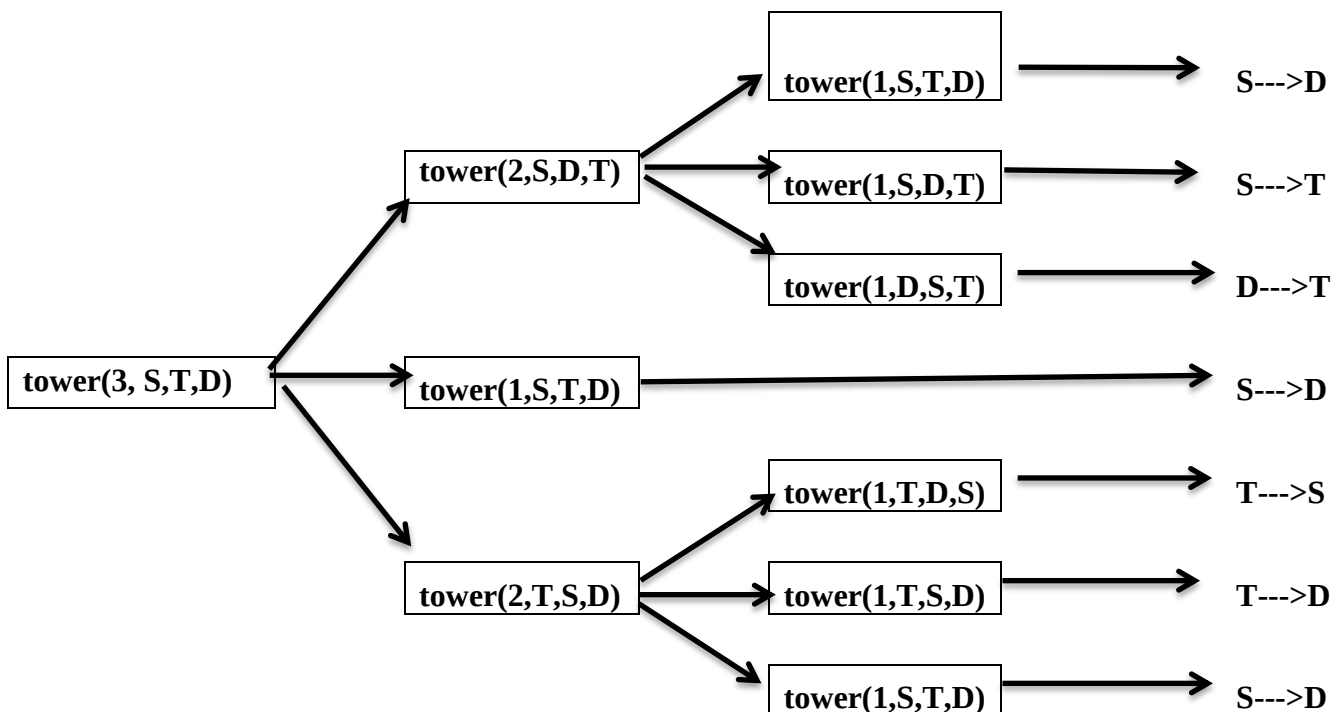
- move n-1 discs from source to temporary
- move nth disc from source to destination
- move n-1 disc from temporary to destination

For example, consider the number of discs = 3. Then, various steps involved in transferring 3 discs from source to destination are shown below –





Recursive tree for towers of Hanoi when n=3



C Program to demonstrate Towers of Hanoi:

```

#include<stdio.h>
int count=0;
void tower(int n, char s, char d, char t)
{
    if(n==1)
    {
        printf("Move disc 1 from %c to %c ", s, d);
        count++; return;
    }
    tower(n-1, s, d, t);
    printf("Move disc %d from %c to %c", n, s,d);
    count++;
    tower(n-1, t, s, d);
}
void main()
{

```

```

int n;
printf("Enter the number of discs");
scanf("%d", &n);
tower(n, "A", "C", "B");
printf("Total number of moves = %d", count);
}

```

Example5: GCD of Two Numbers Using Recursion

The Highest common factor (HCF) of two or more numbers is the greatest number which divides each of them exactly. Greatest Common Measure(GCM) and Greatest Common Divisor(GCD) are the other terms used to refer to HCF. Example: HCF of 60 and 75 = 15 because 15 is the highest number which divides both 60 and 75 exactly.

C program to find gcd of two numbers using recursion

```

#include<stdio.h>

// recursive function to find gcd of two number
int gcd(int a, int b)
{
    if(b!=0)
        return gcd(b, a%b); // general case
    else
        return a; // base case
}

int main()
{
    int n1, n2, result;

    printf("Enter two numbers: ");
    scanf("%d %d",&n1,&n2);

    result = hcf(n1,n2);
    printf("GCD of %d and %d = %d",n1,n2,result);

    return 0;
}

```

In this program, one recursive function gcd is defined which takes two integer arguments and returns int data type. From the main function, the recursive function is called with user-entered value. The parameters of the gcd function hold the value of arguments. Later in the recursive function gcd, through general case gcd(b, a%b) again the function is called until b!=0; Finally, the result is passed back to the main function, and printf() function displayed the result to the screen.

STACK OVERHEADS IN RECURSION

Stack is used to store the current value and return address of the function call. At each recursive call, the stack is pushed to save the necessary values. When a recursive function is invoked, its parameters are stored in a data structure called activation records. Every time the recursive function is invoked, a new activation record is generated and stored in the memory. These activation records are stored in the special stack called the recursive stack. These activation records are deleted when the function execution is completed.

when a recursive function is invoked, it generates the activation records for different values of the recursive function hence, extending the recursion stack size. When the recursive function execution is completed one by one its activation records get deleted hence, contracting the recursive stack size. The recursive stack size depends on the number of activation records created and deleted.

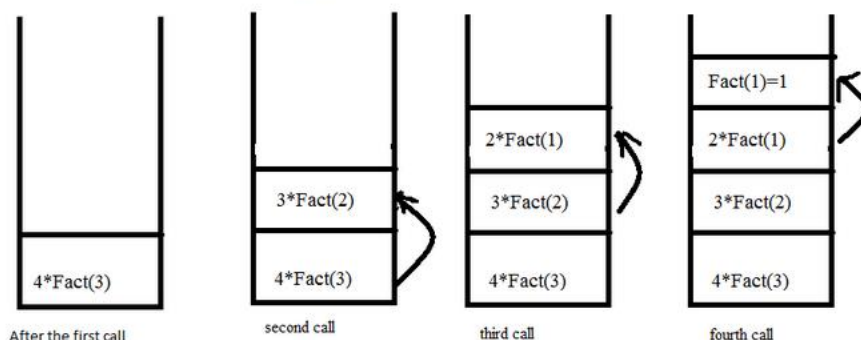
If you analyze the address of local variables of the recursive function, you will get two important results: the depth of recursion and the stack overheads in recursion. Since local variables of the function are pushed into the stack when the function calls another function, by knowing the address of the variable in repetitive recursive call, you will determine how much information is pushed into the stack. For example, the stack could grow from top to bottom, and the local variable *j* gets the address 100 in the stack in the first column. Suppose stack overheads are 16 bytes; in the next call *j* will have the address 84, in the call after that it will get the address 68. That is a difference of 16 bytes. The following program uses the same principle: the difference of the address in consecutive calls is the stack overhead.

let's consider the following function to calculate the factorial of a number:

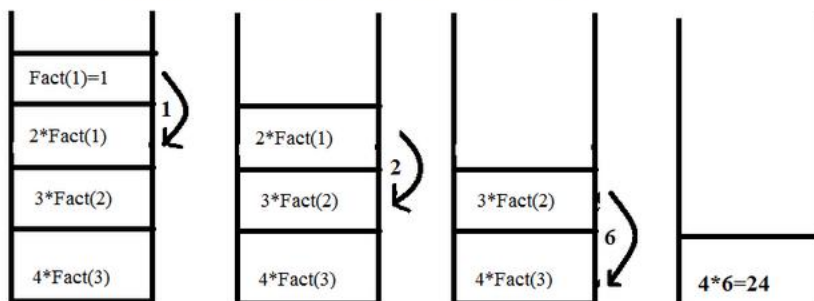
```
function factorial(n) {
  if (n == 0) {
    return 1;
  } else {
    return n * factorial(n - 1);
  }
}
```

Let $n=4$, the below diagram explains the usage of stack calculate 4!

When function call happens previous variables gets stored in stack



Returning values from base case to caller function

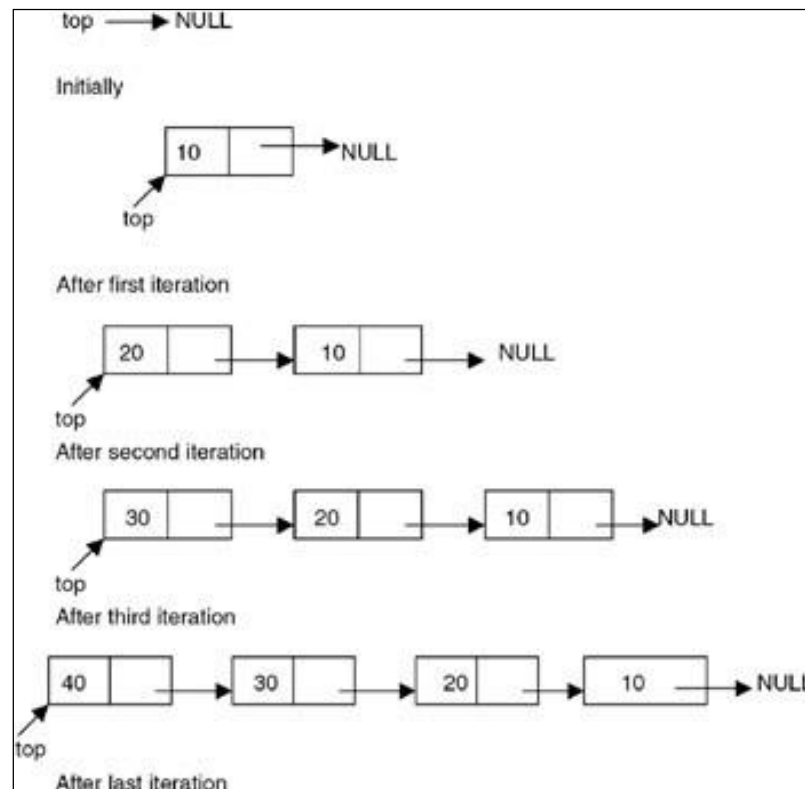


Implementation of a Stack Using Linked Representation

Initially the list is empty, so the top pointer is `NULL`. The push function takes a pointer to an existing list as the first parameter and a data value to be pushed as the second parameter, creates a new node by using the data value, and adds it to the top of the existing list. A pop function takes a pointer to an existing list as the first parameter, and a pointer to a data object in which the popped value is to be returned as a second parameter. Thus it retrieves the

value of the node pointed to by the top pointer, takes the top point to the next node, and destroys the node that was pointed to by the top.

If this strategy is used for creating a stack with the previously used four data values: 10, 20, 30, and



40, then the stack is created as shown in Figure 5.2.

Figure 5.2: Linked stack.

Program

A complete C program for implementation of a stack using the linked list is given here:

```
# include
<stdio.h> #
include <stdlib.h>
struct node
{
    int data;
    struct node *link;
};

struct node *push(struct node *p, int value)
{
    struct node *temp;
    temp=(struct node *)malloc(sizeof(struct node));
    /* creates new node
       using data value
       passed as parameter
       */
    if (temp==NULL)
```

```

{
    printf("No Memory
    available Error\n");
    exit(0);
}

temp-
>data
=
value;
temp-
>link
= p;

p =
tem
p;
retu
rn(p
);
}

struct node *pop(struct node *p, int *value)
{
    struct node
    *temp;
    if (p==NULL)
    {
        printf(" The stack is empty can not pop
        Error\n"); exit(0);
    }

    *value
    = p-
    >data;
    temp =
    p;

    p =
    p-
    >li
    nk;
    free
    (tem
    p);
    retu
    rn(p
    );
}

```



```

void main()
{
    struct node
    *top =
    NULL; int
    n,value;
    do
    {
        do
        {
            printf("Enter the element to be
            pushed\n"); scanf("%d",&value);

            top =
            push(top,value);
            printf("Enter 1 to
            continue\n");
            scanf("%d",&n);
        } while(n == 1);

        printf("Enter 1 to pop
        an element\n");
        scanf("%d",&n);
        while( n == 1)
        {
            top = pop(top,&value);
            printf("The value popped is %d\n",value);

            printf("Enter 1 to pop an
            element\n");
            scanf("%d",&n);

        }

        printf("Enter 1 to
        continue\n");
        scanf("%d",&n);
    } while(n == 1);
}

```

Input and Output

```

Enter the element to be pushed
10
Enter 1 to continue
1
Enter the element to be pushed
20
Enter 1 to continue
0

```

Enter 1 to pop an element

1

The value
popped is

20 Enter 1

to pop an
element

1

The value
popped is

10 Enter 1

to pop an
element

0

Enter 1 to continue

1

Enter the element to be pushed

30

Enter 1 to continue

1

Enter the element to be pushed

40

Enter 1 to continue

0

Enter 1 to pop an element

1

The value popped is 40

Enter 1 to pop an element

0

Enter 1 to continue

1

Enter the element to be pushed

50

Enter 1 to continue

0

Enter 1 to pop an element

1

The value
popped is

50 Enter

1 to pop

an

element

1

The value
popped is

30 Enter

1 to pop

an

element

0
Enter 1 to continue
0

Queue

A Queue Data Structure is a fundamental concept in computer science used for storing and managing data in a specific order.

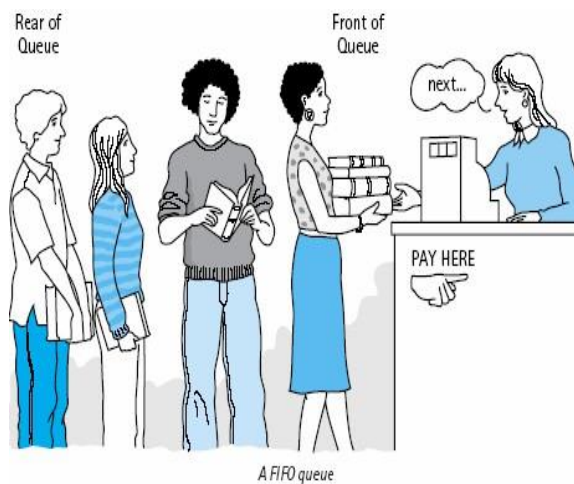
It follows the principle of "First in, First out" (FIFO), where the first element added to the queue is the first one to be removed.

Applications of Queue in computer science:

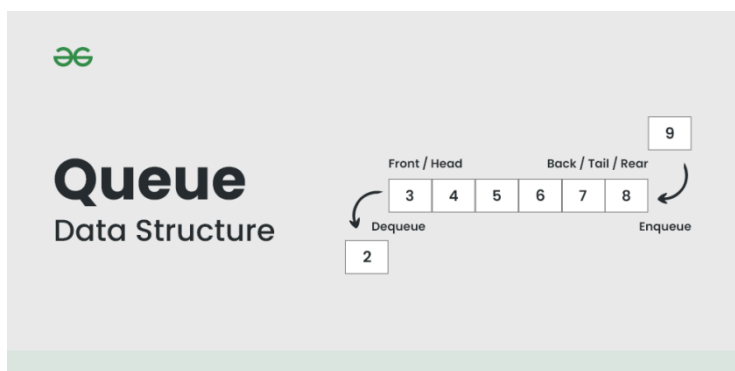
- It is used as a buffer in computer systems where we have speed mismatch between two devices that communicate with each other. For example, CPU and keyboard and two devices in a network
- Queue is also used in Operating System algorithms like CPU Scheduling and Memory Management, and many standard algorithms like Breadth First Search of Graph, Level Order Traversal of a Tree.

Real life example for Queue

The first one in line is the first one to be served.



Definition: A *queue* is also a list of elements with insertions permitted at one end—called the rear, and deletions permitted from the other end—called the front. This means that the removal of elements from a queue is possible in the same order in which the insertion of elements is made into the queue. Thus, a queue data structure exhibits the *FIFO* (*first in first out*) property.



FIFO Principle in Queue:

FIFO Principle states that the first element added to the Queue will be the first one to be removed or processed. So, Queue is like a line of people waiting to purchase tickets, where the first person in line is the first person served. (i.e. First Come First Serve).

Basic Terminologies of Queue:

- **Front:** Position of the entry in a queue ready to be served, that is, the first entry that will be removed from the queue, is called the front of the queue. It is also referred as the head of the queue.
- **Rear:** Position of the last entry in the queue, that is, the one most recently added, is called the rear of the queue. It is also referred as the tail of the queue.
- **Size:** refers to the maximum number of elements the queue can hold.

Queue Operations:

Enqueue: Adds an element to the end (rear) of the queue. If the queue is full, an overflow error occurs.

Dequeue: Removes the element from the front of the queue. If the queue is empty, an underflow error occurs.

Peek/Front: Returns the element at the front without removing it.

Size: Returns the number of elements in the queue.

isEmpty: Returns true if the queue is empty, otherwise false. isEmpty condition of Queue data structure is :

`rear == -1 || front == rear`

isFull: Returns true if the queue is full, otherwise false. isFull condition of Queue is:

`rear == max-1`

Applications of Queue Data Structure

Application of queue is common. In a computer system, there may be queues of tasks waiting for the printer, for access to disk storage, or even in a time-sharing system, for use of the CPU. Within a single program, there may be multiple requests to be kept in a queue, or one task may create other tasks, which must be done in turn by keeping them in a queue.

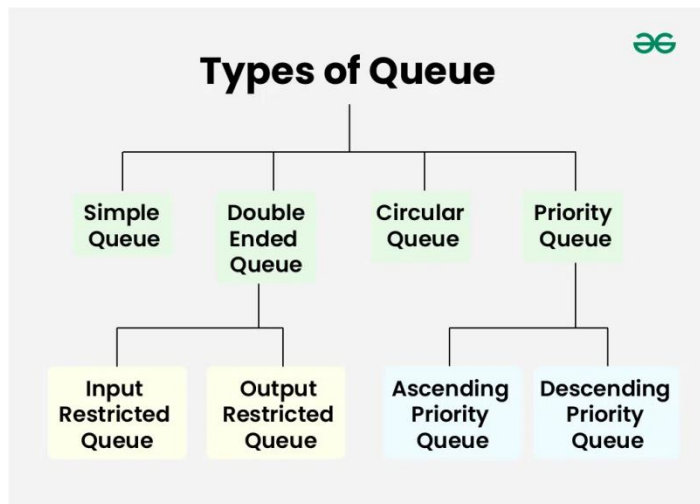
- A Queue is always used as a buffer when we have a speed mismatch between a producer and consumer. For example keyboard and CPU.
- Queue can be used where we have a single resource and multiple consumers like a single CPU and multiple processes.
- In a network, a queue is used in devices such as a router/switch and mail queue.
- Queue can be used in various algorithm techniques like Breadth First Search, Topological Sort, etc.

Types of Queues:

There are four different types of queues in data structures:

- **Simple Queue**
- **Circular Queue**
- **Priority Queue**
- **Double-Ended Queue (Deque)**

Simple Queue



Simple Queue : is a linear data structure that follows the First-In-First-Out (FIFO) principle, where elements are added to the rear (back) and removed from the front (head).

- Ordered collection of comparable data kinds.
- Queue structure is FIFO (First in, First Out).
- When a new element is added, all elements added before the new element must be deleted to remove the new element.

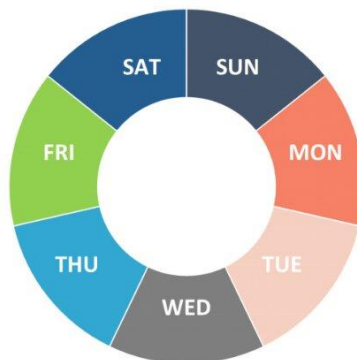


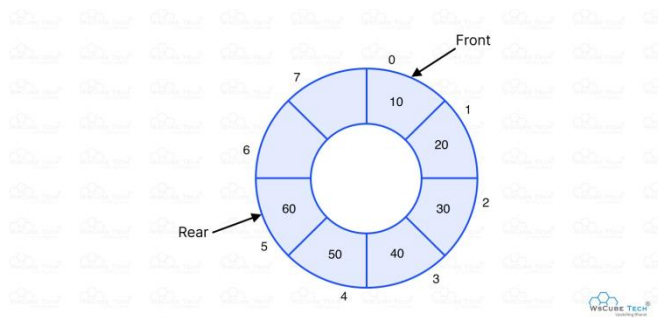
Applications of simple Queue:

- **Resource Allocation:** Simple Queues are useful for resource allocation in operating systems that manage resource requests such as CPU, memory, and I/O devices.
- **Batch Processing:** Queues accommodate batch jobs, for instance, tasks like data processing or rendering images, which are queued up for sequential execution.
- **Message Buffering:** It helps to buffer the message in communication systems to ensure a smooth data flow among processes.

Circular Queue: A circular queue is a special case of a simple queue in which the last member is linked to the first, forming a circle-like structure.

- The last node is connected to the first node.
- Also known as a **Ring Buffer**, the nodes are connected end to end.
- Insertion takes place at the front of the queue, and deletion at the end of the queue.
- **Example of circular queue application:** Insertion of days in a week.





Applications of Circular Queue:

- **CPU Scheduling:** Used in operating systems to manage processes.
- **Data Buffering:** Frequently used in data streaming and buffering applications.
- **Simulation Systems:** Helpful in simulating real-world systems and processes, e.g., traffic flow control.

Priority Queue: In a priority queue, the nodes will have some predefined priority in the priority queue. The node with the least priority will be the first to be removed from the queue. Insertion takes place in the order of arrival of the nodes.

Applications of priority queue:

- Dijkstra's shortest path algorithm
- Prim's algorithm
- Data compression techniques like Huffman code

The diagram below shows how an application uses a priority queue for the items the user consumes.



Types of Priority Queue:

1. **Ascending Priority Queue:** Element can be inserted arbitrarily but only smallest element can be removed. For example, suppose there is an array having elements 4, 2, 8 in the same order. So, while inserting the elements, the insertion will be in the same sequence but while deleting, the order will be 2, 4, 8.
2. **Descending priority Queue:** Element can be inserted arbitrarily but only the largest element can be removed first from the given Queue. For example, suppose there is an array having elements 4, 2, 8 in the same order. So, while inserting the elements, the insertion will be in the same sequence but while deleting, the order will be 8, 4, 2.

Priority Queue Applications

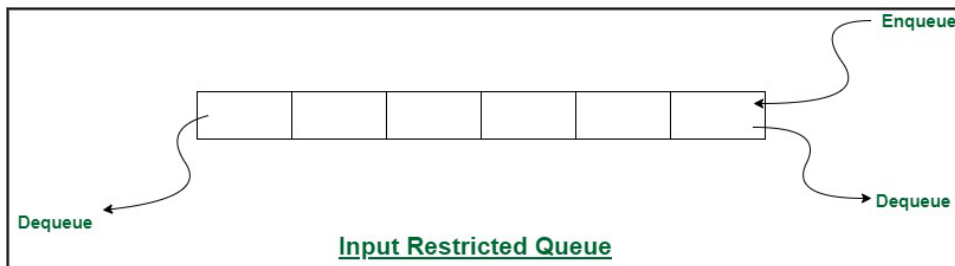
- **Data Compression:** In methods like Huffman coding, priority queues organize characters based on how often they appear, which helps reduce file sizes.
- **Event Simulation:** Priority queues manage events in simulations, making sure that events happening sooner are processed before later ones.
- **Top-k Elements:** They can keep track of the top-k items coming from a data stream, allowing rapid access to the most important ones.

Deque (Double Ended Queue): A double-ended queue, often abbreviated as deque (pronounced "deck"), is a type of data structure that allows elements to be added or removed from both the front and the rear, making it a versatile option for various applications. Unlike regular queues that follow the FIFO principle, deques do not have this restriction.

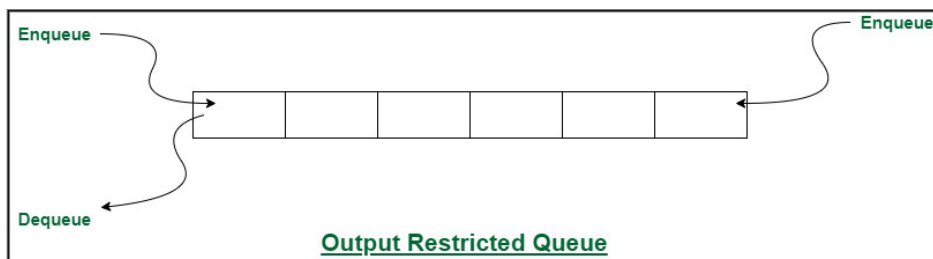


Types of Deque:

1. **Input restricted Queue:** In this type of Queue, the input can be taken from one side only(rear) and deletion of elements can be done from both sides(front and rear). This kind of Queue does not follow FIFO(first in first out). This queue is used in cases where the consumption of the data needs to be in FIFO order but if there is a need to remove the recently inserted data for some reason and one such case can be irrelevant data, performance issue, etc.



2. **Output restricted Queue:** In this type of Queue, the input can be taken from both sides(rear and front) and the deletion of the element can be done from only one side(front). This queue is used in the case where the inputs have some priority order to be executed and the input can be placed even in the first place so that it is executed first.



Deque Applications:

- **Undo/Redo Functions:** Deques are useful in applications for keeping track of actions so users can go back or forward through their history.
- **Store Web Browser History:** In a web browser, deques can be used to easily store the visited pages to add new pages and remove old ones.
- **Graph Traversal:** In algorithms like Breadth-First Search (BFS), deques help efficiently manage which nodes to explore next.
- **Palindrome Checking:** Deques can check if a word or phrase reads the same backwards as forward by comparing letters from both ends.

Array Representation of Queue

Simple Queue Array implementation :

For implementing the queue, we only need to keep track of two variables: front and size. We can find the rear as $\text{front} + \text{size} - 1$.

The Enqueue operation is simple, we simply insert at the end of the array. This operation takes $O(1)$ time

The Dequeue operation is costly as we need remove from the beginning of the array. To remove an item, we need to move all items one position back. Hence this operations takes $O(n)$ time.

If we do otherwise that insert at the begin and delete from the end, then insert would become costly.

Basic Queue Operations Algorithms using array:

Enqueue Operation

Below are the steps to enqueue (insert) data into a queue

- Check whether the queue is full or not.
- If the queue is full – print the overflow error and exit the program.
- If the queue is not full – increment the rear pointer to point to the next empty space.
- Else add the element in the position pointed by Rear.
- Return success.

Algorithm for Enqueue Operation

procedure enqueueer (data)

if queue is full

return overflow

endif

rear $\leftarrow$ rear + 1

queue[rear] $\leftarrow$ data

return true

end procedure

Dequeue Operation

Below are the steps to perform the dequeue operation

Check whether the queue is full or not.

If the queue is empty – print the underflow error and exit the program.

If the queue is not empty – access the data where the front is pointing.

Else increment the front pointer to point to the next available data element.

Return success.

Algorithm for Dequeue Operation

procedure dequeue

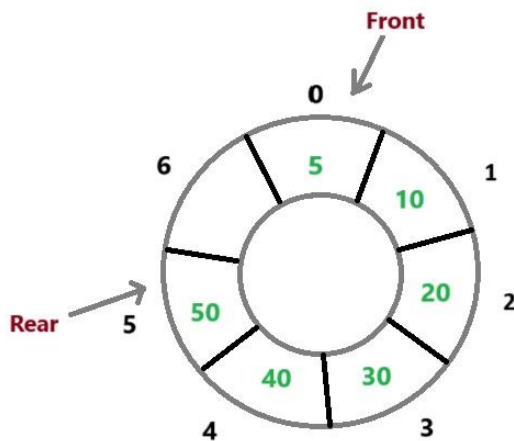
if queue is empty

return underflow

end if data = queue[front] front $\leftarrow$ front + 1 return true end procedure

Circular Queue Array implementation :

Following is the representation of a circular queue, where the front is the index of the first element, and the rear is the index of the last element.



Operations on Circular Queue using Array:

There are mainly four operations that can be performed on a circular queue:

- Enqueue: Insert an element into the circular queue.
- Dequeue: Delete an element from the circular queue.
- Front: Get the front element from the circular queue.
- Rear: Get the last element from the circular queue.

Algorithm for Enqueue Operation

Following are the steps to perform the enqueue operation on a circular queue:

- Initialize an array or any data structure to store the elements of the circular queue.
- Initialize two variables front and rear.
- Check if the circular queue is full.
- If it is not full, and if the front is -1, set the front to 0.
- Increment the rear by 1 and store it in the rear index.
- Update the rear index using $\text{rear} = (\text{rear} + 1) \% \text{SIZE}$.

procedure Enqueue {

```

if(isFull())
    print( Queue is full)
else {
    if(front == -1) front = 0
    rear = (rear + 1) % SIZE
    items[rear] = element
    }

```

}

Algorithm for Dequeue Operation

Following are the steps to perform the dequeue operation on a circular queue:

- Check if the circular queue is empty.
- If the queue is not empty, store the element at the front index.

- If the front and rear are equal, set the front and rear to -1.
- Else, increase the front index by 1 using the formula $\text{front} = (\text{front} + 1) \% \text{SIZE}$.
- At last print the deleted element.

```

procedure Dequeue {
    if(isEmpty())
        print( Queue is empty)
    else
    {
        element = items[front];
        if (front == rear)
            front = rear=-1;
        else
            front = (front + 1) % SIZE;
    }
    printf( Deleted element)
    return(element)
}

```

Refer lab programs for the following topics:

Implement a stack using an array

Implement a queue using a linked list

Evaluate a postfix expression using a stack